

黄成君

15927642709 huangcj.work@foxmail.com 男 26岁

MySQL JDBC Contributor

教育经历

武汉纺织大学（一本） 计算机科学与技术 本科

2017-09 ~ 2021-06

- 在校期间，取得优异成绩，保持GPA排名前5%，获得两次一等奖学金、一次二等奖学金。
- 利用业余参加各类比赛取得显著成绩，包括蓝桥杯全国二等奖、全国大学生数学建模竞赛省二等奖。

工作经历

东方财富集团-国际证券业务部

2021-07 ~ 2024-08

交易开发团队 服务端开发工程师 上海

- 负责金融业务后台系统架构设计与核心代码编写。拆解外采系统的业务逻辑，重构为采用 **Java**、**Spring Boot**、**MySQL**、**redis**、**zookeeper**、**netty** 技术栈的自研系统。从事清结算系统、自研网关、自研数据中间件、交易链路等系统的开发及优化工作。
- 发现 MySQL JDBC 存在优化空间，并为 MySQL JDBC 贡献代码。

作业帮

2025-07 ~ 至今

快对AI 服务端开发工程师 武汉

- 负责广告模块的开发 qps。
- 主导建立了一套深度结合 AI 辅助开发的工作流。通过在 codebase 建立可持续集成的知识库，降低了 AI 修改复杂工程代码时出现幻觉的几率。

项目经历

交易链路性能优化

项目背景：

交易链路需要根据非速动证券进行购买力校验。非速动证券的计算需要获取全量用户的资金证券的实时状态。原逻辑通过定时任务查询多张**百万级**表计算，接口耗时 20s。交易链路只能获取延迟数据。业务评估时效性低于预期，且 IO 占用过高容易引发雪崩。急需优化。

技术栈：SpringBoot、MyBatis、MySql、Redis、ZooKeeper、Netty

- 通过静态数据 + 增量流水汇总计算得出当前状态。以流水号为游标实现**游标查询**，数据库单次增量获取小于 100 条，IO 占用减小 99%。同时引入定时任务计算防止流水积压过长，实时性从20秒提高到**毫秒级**。
- 降级规则**，开启线程异步计算，主线程等待时间达到阈值后直接返回上次缓存的结果和时间戳，防止长尾效应。
- 使用 **ReentrantLock + 时间戳版本号** 的方式代替 synchronized 方法，进一步提高并行度。db 调用从 100qps->50qps，吞吐量从 50 -> 2500。

突出点：

- 创新性架构**：静态预计算 + 增量流处理模式，突破传统定时任务的性能瓶颈；
- 精细化熔断**：多级熔断策略覆盖计算超时、资源过载等场景，保障核心交易链路；
- 极致性能优化**：通过游标查询、**锁粒度控制**、缓存预热等组合拳，实现毫秒级响应。

国际证券柜台 统一清算平台

项目背景：

负责将原有外采基于 Oracle+C++ 的清算系统重构为 SpringBoot+MySQL 架构的自研系统，实现日间交易的高效清算与数据归档。系统支持百万级用户交易的清算，确保资券准确交收，并具备异常订单的兜底处理能力。

技术栈：SpringBoot、MyBatis、MySql、Redis、ZooKeeper

- 内存治理与性能优化**：通过 visualvm 分析 **JVM 内存快照**定位到 BigDecimal 实例膨胀问题，优化 JDBC 底层类型转换逻辑。系统内存占用降低约 70%。
- 死锁治理**：采用多种方式避免死锁的产生。1. 调整股票上账顺序，**避免循环等待**。从而防止公司仓位表更新出现死

- 锁。2. **减小锁范围**，上账时将原本一事物多用户的模式修改为一事物一用户。
- **测试体系升级**：引入 Testcontainers 构建**容器化测试环境**，通过并行执行（JUnit Parallel）+ 容器复用，单次测试耗时从 6 小时缩短至 10 分钟。
- **订单对账重构**：原始对账逻辑随着业务复杂度提升，不同业务进入的订单对账逐渐演化为烟囱式硬编码。引入**责任链模式**重构，**代码复用率提升 60%**，新业务接入时间减少一半。

突出点：

- **分布式清算+分片逻辑设计**：基于 zk 实现**服务注册和任务分片**；综合考虑MySQL索引结构，根据用户Id 的中间几位建立**函数索引**，在数据均匀性和db读写性能间达到平衡。**使用 redis 预热缓存减轻主库压力**。无法分治处理的任务，拆分成低内存占用单机任务 + 分布式任务的模式。2台实例下百万用户清算耗时从 40min 减少到 25min（**减少35%**）。
- **任务幂等控制**：通过**乐观锁**实现任务幂等控制。采用类似 **TCC 模式**，令每个任务实现回滚逻辑，使任务支持回滚、重跑等异常处理。上账过程参考 **RedoLog**，实现两步上账（**簿记+上账**），解决上账出现异常后无法回滚的问题。
- **一致性保障**：设计**双重对账逻辑**（**订单对账+存量证券对账**）。从多个维度保障数据一致性。针对异常场景设计补单功能兜底，通过人工介入实现最终一致性。
- **TDD + 双路清算，保障迁移前后逻辑一致性**：基于历史数据库，通过数据清洗构造包含台风天、半日市、长假等场景的千万级回归数据集。结合TDD实现开发阶段的验证。试运行期间，通过同步数据至自研环境 + 双路清算校验，完成线上环境验证。**百万用户迁移成功率 100%**。
- **冷热数据分离**：清算完成后将当前表数据归档到历史表，实现**业务层面分表**，减小热点数据总量。历史表通过**分区表**实现高效查询和修改，分区后归档任务关键路径耗时从10min 减少到 10s。

内容生产平台多级缓存治理

~ 至今

项目背景

拍照翻译 C 端链路中存在两层缓存：资源中台缓存，以及服务内部的 Redis 旁路缓存。当资源更新时需要同时删除两层缓存，但由于资源中台历史版本只支持缓存重建，在数据库存在主从延迟的情况下，会把旧数据重新写入缓存，导致上下游缓存不一致问题偶现，在高并发场景下尤为突出。

技术栈：Golang、MySql、Redis、RocketMQ、Elasticsearch

解决方案

- **删除替代重建**：将资源更新时的逻辑从“重建缓存”改为“删除缓存”，避免因主从延迟将旧数据写回缓存的问题。
- **解耦刷新与删除**：通过 RocketMQ 解耦数据库更新与缓存删除操作，降低数据库更新与缓存处理之间的耦合度，减少链路抖动。
- **延迟删除策略**：利用 MQ 的**延迟消息**机制，在主从延迟消除后再触发缓存删除，保证缓存与数据库的一致性，即使在高并发场景下也能稳定工作。
- **两层缓存协同**：在业务侧统一封装缓存处理逻辑，确保 Redis 旁路缓存与资源中台缓存能够在一致的时机失效，避免出现前后端数据不一致。

其他经历

待业

2024-09 ~ 2025-07

- Gap 期间，以全面自我突破为目标，通过技术深耕、项目实战与健康管理和并行，实现**专业能力与身体素质的双重进阶**。
- **技术能力提升**：系统研读技术书籍，完成系列知识总结与实践文档；于 Coursera 学习 **AI 模型端技术**，成功应用于《债券自动打新》项目。
 - **项目实践**：1. 借助 Cursor AI 编程工具完成《docker volume backup》项目，实现容器数据安全备份。2. 开发债券打新工具，实现自动打新债券。
 - **个人管理**：制定科学计划，在 Gap 期间达成**减重20斤**目标。开发了适合自己的减重分餐应用。
 - 其他：2024.10 - 11月考取驾照。 2025.2月底做了手术，目前身体已经完全恢复，可以适应高强度工作。

专业技能

- 参与开源项目，为 MySQL JDBC 贡献过源码。
- 熟悉常见限流算法，创新性为自研网关设计了**优化的滑动窗口限流算法**，使用 **redis + lua 脚本**实现分布式限流，限流判断时间开销减小 50%

发布新功能时间平均减少 50%

- 熟悉 java 语言开发，深入理解 JDK 集合，研究过常用类源码。
- 熟悉 MySQL 使用和原理，**熟悉索引、事务、日志、锁**等机制，有大量 SQL 调优经验，了解读写分离、分库分表。
- 熟悉 Redis 使用和原理，持久化机制等，熟悉高可用方案，熟悉缓存雪崩、穿透、击穿解决方案。
- 熟悉 SpringBoot、Mybatis 等框架，能够运用并独立开发项目，熟悉AOP、IOC等原理。
- 熟悉常用**设计模式**，如责任链模式、工厂模式并运用到实际开发当中。
- 了解 Spring Cloud 微服务架构，了解 GateWay、Nacos、的基本使用。了解 Kafka
- 了解 **Docker** 及其相关组件和容器化技术。
- 具备 Vue 和 React 的使用经验，能够独立进行前端开发工作。
- 有**AI 模型端**相关经验，有将 AI 应用于私有项目的经验。
- 熟悉主流 **AI 编程工具**，具备基于 AI 辅助从 0 开发完整产品的实践经验。